



# High Performance Computing

Dr.Bheemappa H  
IIIT, Sricity  
bheemappa.h@iiit.in

# Outline

2

- Course Details
- Syllabus
- Evaluation Plan
- Reference Materials
- SUPERCOMPUTERS

- What is computer Program How Data Represented?
  - Algorithm and data structure.
  - Programming Language: Standard Notation for writing programs
    - Example : C, Java, Intel assembly language
      - **Machine Language(IA32).**
  - Program translators : Compiler - GCC
    - What does GCC do?
- Binary
  - Base2/0-1/bit

## Improving of Computer Performance :

4

- Innovative manufacturing and advancement of VLSI.
- Exploitation of some form parallelism
  - ILP
    - Pipelining (IE=F/D/E/M/)
    - Out of order execution
    - Superscalar
- Thread Level Parallelism
  - Multiple thread on one processor
  - SMT- Hardware Multithreading.
  - Multithreading on Multiple processor.
  - Example : Loop
- Process Level Parallelism
  - Different processes can be executed in parallel on Multiple process
    - SMP
    - DSM

# Syllabus

## **Module 1: Introduction:**

Motivating Parallelism, Scope of Parallel Computing, Introduction to HPC: Parallel Programming Platforms; Implicit Parallelism: Trends in Microprocessor Architectures, Limitations of Memory System Performance, Dichotomy of Parallel Computing Platforms, Physical Organization of Parallel Platforms; .

## **Module 2: Parallel Computer Memory Architectures:**

Measures of Parallel Algorithms; Analytical Modeling of Parallel Programs: Sources of Overhead in Parallel Programs, Performance Metrics for Parallel Systems, the Effect of Granularity on Performance; Parallel Platforms: Models (SIMD, MIMD, SPMD) , Communication (Shared Address Space vs. Message Passing)

## **Module 3 Threads :**

Thread Basics: Why Threads? The POSIX Thread API, Thread Creation and Termination, Synchronization Primitives in Pthreads, Controlling Thread and Synchronization Attributes, Thread Cancellation, Composite Synchronization.

## **Module 4: OpenMP**

Tips for Designing Asynchronous Programs, OpenMP: A Standard for Directive Based Parallel Programming.

## **Module 5 Distributed Memory Parallel Programming**

The Building Blocks: Send and Receive Operations, MPI: the Message Passing Interface, Topologies and Embedding, Overlapping Communication with Computation, Collective Communication and Computation Operations, Groups and Communicators.

## **Module 5 GPGPU:**

The Age of Parallel Processing, Central Processing Units, The Rise of GPU Computing, A brief history of GPUs, Early GPU computing; CUDA: What is CUDA architecture, using the CUDA architecture, Applications of CUDA, Medical Imaging, Computational Fluid Dynamics, Environmental Science; Introduction to CUDA C: A First Program, Hello world, A kernel call, Passing parameters, Querying devices, using device properties; Parallel Programming in CUDA C: CUDA parallel programming, Summing vectors, A fun example.

## TEXT BOOKS

6

- John Hennessy and David Patterson, Computer Architecture - A Quantitative Approach. 5ed. Morgan Kaufmann.
- Ananth Grama, Vipin Kumar, Anshul Gupta, George Karypis, Introduction to Parallel Computing, Addison-Wesley, 2003
- Wen-Mei W Hwu, David B Kirk, Programming Massively Parallel Processors A Hands-on Approach, Morgan Kaufmann, 3e.
- William Dally and Brian Towles. Principles and Practices of Interconnection Networks, MK, 2004

# EVALUATION PLAN

7

| Mid-Evaluation | End Evaluation | Class/Tutorial-Participat | Quiz | Mini Project | Total |
|----------------|----------------|---------------------------|------|--------------|-------|
| 25             | 40             | 5                         | 10   | 20           | 100   |

| Week    | Components |
|---------|------------|
| week 4  | P-0/A-0    |
| week 6  | Quiz-1     |
| Week-7  | P1         |
| week 8  | Mid-Exam   |
| week 15 | Quiz-2     |
| Week15  | P1         |

# Week-1:Class-2

# M1:

9

- Introduction
- Introduction to HPC
- Motivating Parallelism
- Scope of Parallel Computing
- Parallel Programming Platforms
  - Implicit Parallelism: Trends in Microprocessor Architectures
  - Limitations of Memory System Performance
- Dichotomy of Parallel Computing Platforms
  - Control Structure of Parallel Platforms
  - Communication Model of Parallel Platforms
- Physical Organization of Parallel Platforms
  - Architecture of an Ideal Parallel Computer
  - Interconnection Networks for Parallel Computers

# Historical Background

10

## Two Major Stages of Development

- Mechanical : before 1945
- Electronic : after 1945
  - 1st Gen 1945-54 : Vacuum Tubes/assembly language/IBM 701 )
  - 2nd Gen 1955-64 : Transistor/compiler/IBM 7090)
  - 3rd Gen 1965- 97: IC-SSI,MSI,**I/Multiprogramming/time sharing/IBM 360/70**
  - **4th Gen - 197-90** : IC-VLSI/Multiprocessor/HLL/Parallel Processing(home))
  - **5th Gen-1991- Till** : (IC-ULSI/Parallel Processing (hetero))/Cray MPP)

# M1:

11

- Introduction to HPC:
- Motivating Parallelism
- Scope of Parallel Computing
- Parallel Programming Platforms
  - Implicit Parallelism: Trends in Microprocessor Architectures
  - Limitations of Memory System Performance
- Dichotomy of Parallel Computing Platforms
  - Control Structure of Parallel Platforms
  - Communication Model of Parallel Platforms
- Physical Organization of Parallel Platforms
  - Architecture of an Ideal Parallel Computer
  - Interconnection Networks for Parallel Computers

# Introduction to HPC

12

- Elements of Modern Computer
  - Hardware
  - System Software
  - Application Programs
    - Functionality of Processor is characterized by **Instruction Set**.
    - **What is Architecture and Organization.**
      - **Architecture : Short form of ISA.(Programmer view of Processor)**
        - Data transfer, Add, Mul, sub etc. , register, instruction set etc.
      - **Organization: High level Design, : Caches , Pipeline , controlling Unit.**

# Introduction to HPC

13

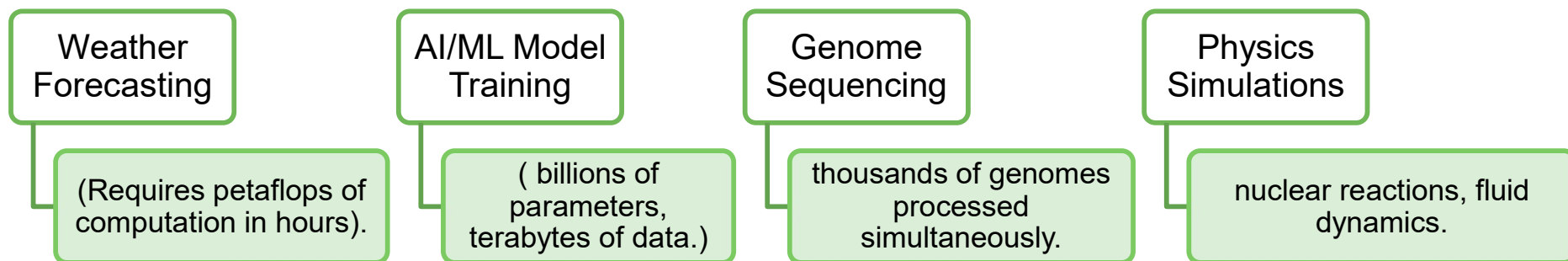
## High-Performance Computing (HPC) ?

**Aggregation of computing power** to solve the problem that are too big or too slow for regular computers.

Example of HPC Systems.

- Supercomputers
- High-end clusters in data centres
- Hybrid CPU–GPU systems

### What are the Real-world scenarios where serial computing struggles ?



<https://bheemhh.github.io/#/HPC26>

- What is high performance computing (HPC)?
  - use of the most **efficient algorithms** on computers capable of the highest performance to solve the most demanding problems
  - HPC main resources are : CPU, RAM, GPU
- Why HPC?
  - Large problems – spatially/temporally
    - Large problems – spatially/temporally
      - Weather forecasting; protein folding; turbulence simulations/CFD; aerospace structures; Full-body simulation/
      - Earthquake simulation.

# SUPERCOMPUTERS

15

- The term is commonly applied to the fastest high-performance systems available at any given time.
- Computers are used Primarily for **scientific and engineering** work requiring exceedingly **high-speed computations**.
- Top 500 Super computer: <https://en.wikipedia.org/wiki/TOP500>

|                         |                                                                                  |                                       |                             |         |                                                       |      |                        |
|-------------------------|----------------------------------------------------------------------------------|---------------------------------------|-----------------------------|---------|-------------------------------------------------------|------|------------------------|
| HPE Cray EX235a         | 591,872<br>(9,248 × 64-core<br>Optimized 3rd<br>Generation EPYC 64C<br>@2.0 GHz) | 36,992 × 220 AMD<br>Instinct MI250X   | Slingshot-11                | HPE     | Oak Ridge National<br>Laboratory<br>United States     | 2022 | Linux (HPE Cray<br>OS) |
| Supercomputer<br>Fugaku | 7,630,848<br>(158,976 × 48-core<br>Fujitsu A64FX<br>@2.2 GHz)                    | 0                                     | Tofu interconnect<br>D      | Fujitsu | RIKEN Center for<br>Computational<br>Science<br>Japan | 2020 | Linux (RHEL)           |
| HPE Cray EX235a         | 150,528<br>(2,352 × 64-core<br>Optimized 3rd<br>Generation EPYC 64C<br>@2.0 GHz) | 9,408 × 220 AMD<br>Instinct MI250X    | Slingshot-11                | HPE     | EuroHPC JU<br>European Union,<br>Kajaani, Finland     | 2022 | Linux (HPE Cray<br>OS) |
| BullSequana<br>XH2000   | 110,592<br>(3,456 × 32-core Xeon<br>Platinum 8358<br>@2.6 GHz)                   | 13,824 × 108<br>Nvidia Ampere<br>A100 | Nvidia HDR100<br>Infiniband | Atos    | EuroHPC JU<br>European Union,<br>Bologna, Italy       | 2022 | Linux                  |

- Measured in **quadrillions** of 64-bit **floating point** operations per **second**.
- The speed of floating-point operations, commonly measured in terms of **FLOPS**,
- **Floating point operations per second (FLOPS, flops or flop/s)** is a measure of **computer performance**,

$$\text{FLOPS} = \text{cores} \times \frac{\text{cycles}}{\text{second}} \times \frac{\text{FLOPs}}{\text{cycle}}.$$

<https://bheemhh.github.io/#/HPC26>

## What is a HPC system and how does it differ from a regular computer?

16

- HPC

- Major components
  - CPU, RAM, GPU
- Several computers connected together in a network (forming a HPC cluster). Each of these computers is referred to as a node in the network.
- HPC clusters is to run resource-intensive and/or parallel task

- Nodes in HPC :

- Two types of nodes on a cluster
  - login nodes (also known as head or submit nodes).
  - compute nodes (also known as worker nodes).

- Job Scheduler

- Users do not have direct access to the *compute nodes* and instead submitting jobs via a *job scheduler*.
- scheduler's role is to manage all these jobs

# High Performance Computers

17

- In 1980s
  - $1 \times 10^6$  Floating Point Ops/sec (Mflop/s)
  - Scalar based
- In 1990s
  - $1 \times 10^9$  Floating Point Ops/sec (Gflop/s)
  - Vector & Shared memory computing
- Today
  - $1 \times 10^{12}$  Floating Point Ops/sec (Tflop/s)
  - Highly parallel, distributed processing, message passing

## Big Data Problem

Too much data



Not enough compute power,  
storage or infrastructure



<https://bheemhh.github.io/#/HPC26>

# Computer Technology

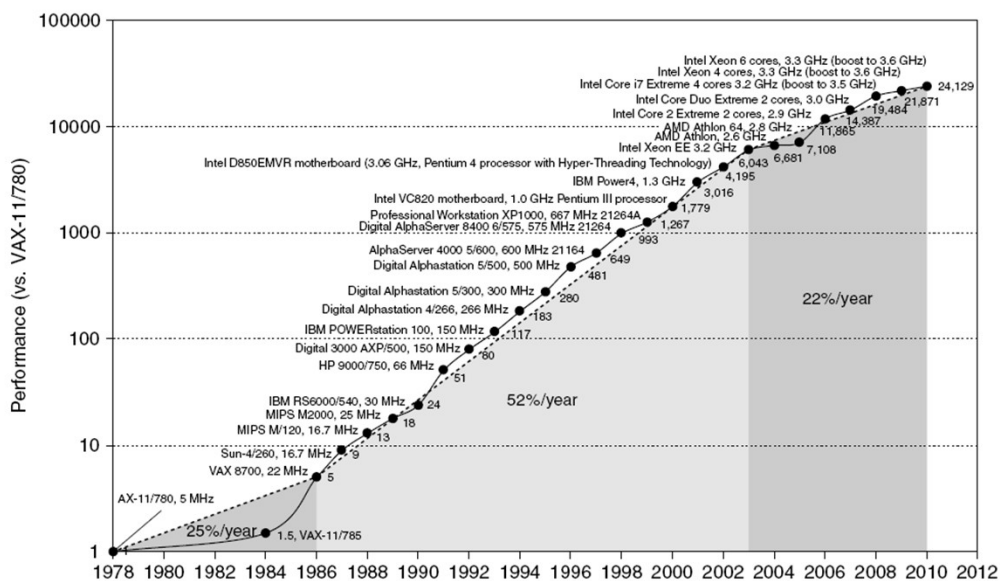
19

- Performance improvements:
  - Improvements in semiconductor technology
    - Feature size, clock speed
  - Improvements in computer architectures
    - Enabled by HLL compilers, UNIX
    - Lead to RISC architectures
  - Together have enabled:
    - Lightweight computers
    - Productivity-based managed/interpreted programming languages
    - SaaS, Virtualization, Cloud
  - Applications evolution:
    - Speech, sound, images, video, “augmented/extended reality”, “big data”

# Moore's Law

20

Gordon Moore (co-founder of Intel) predicted in 1965 that the transistor density of semiconductor chips would double roughly every 18 months.



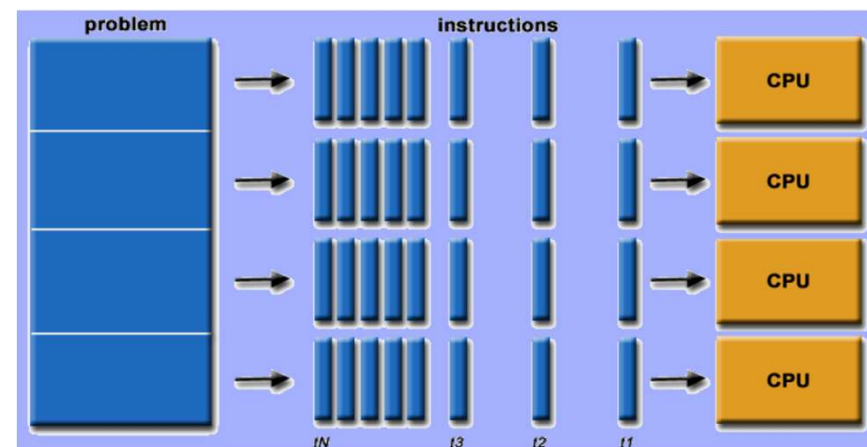
|                                      | 1945      | 2002          |
|--------------------------------------|-----------|---------------|
| Computer                             | ENIAC     | Laptop        |
| Number of vacuum tubes / transistors | 18 000    | 6 000 000 000 |
| Weight (kg)                          | 27 200    | 0.9           |
| Size (m³)                            | 68        | 0.0028        |
| Power (watts)                        | 20 000    | 60            |
| Cost (\$)                            | 4 630 000 | 1 000         |
| Memory (bytes)                       | 200       | 1 073 741 824 |
| Performance (Flops/s)                | 800       | 5 000 000 000 |

<https://bheemhh.github.io/#/HPC26>

# Introduction to Parallel Computing

21

- What is Parallel Computing?
  - “Processing of multiple tasks simultaneously on multiple processor is called parallel processing”.
- Advantages of Parallel Computing



# M1:

22

- Introduction to HPC:
- Motivating Parallelism, Scope of Parallel Computing,
- Parallel Programming Platforms
  - Implicit Parallelism: Trends in Microprocessor Architectures
  - Limitations of Memory System Performance
- Dichotomy of Parallel Computing Platforms
  - Control Structure of Parallel Platforms
  - Communication Model of Parallel Platforms
- Physical Organization of Parallel Platforms
  - Architecture of an Ideal Parallel Computer
  - Interconnection Networks for Parallel Computers

# Motivating Parallelism

23

- The role of parallelism in **accelerating computing speeds** has been recognized for several decades.
- Its role in providing multiplicity of datapaths and increased access to storage elements has been significant in commercial applications
- The scalable **performance and lower cost of parallel platforms** is reflected in the wide variety of applications
- The emergence of standardized parallel programming environments, libraries, and hardware have significantly reduced time to (parallel) solution

# Motivating Parallelism

24

- Computational Speed Argument
  - The logical recourse is to rely on parallelism, both implicit and explicit
  - Most serial (or seemingly serial) processors rely extensively on implicit parallelism.
- Memory/Disk Speed Argument
  - Mismatch in speeds causes significant performance bottlenecks.
  - Parallel platforms provide increased **bandwidth to the memory system**
    - **Principles of locality of data reference and bulk access,**
- Data Communication Argument :
  - Vision of the Internet as one large computing platform has emerged.
  - Any analyses on this data must be performed over the network using parallel techniques

# Scope of Parallel Computing Applications

25

- Applications in Engineering and Design.
- Scientific Applications
  - evolution of galaxies, thermonuclear processes, and the analysis of extremely large datasets from telescopes
- Commercial Applications
  - optimizing business and marketing decisions
- Applications in Computer Systems
  - Network intrusion detection, cryptography
- Applications in Data Science
  - Google File System, Map-reduce

- **Concurrency** is when two tasks can start, run, and complete in overlapping time periods.
- **Parallelism** refers to techniques to make programs faster by performing several computations at the same time.

# M1:

27

- Motivating Parallelism, Scope of Parallel Computing,
- Introduction to HPC:
- Parallel Programming Platforms
  - Implicit Parallelism: Trends in Microprocessor Architectures
  - Limitations of Memory System Performance
- Dichotomy of Parallel Computing Platforms
  - Control Structure of Parallel Platforms
  - Communication Model of Parallel Platforms
- Physical Organization of Parallel Platforms
  - Architecture of an Ideal Parallel Computer
  - Interconnection Networks for Parallel Computers

# Implicit Parallelism: Trends in Microprocessor Architectures

HPC-M25|IIITS

28

- Conventional architectures coarsely comprise of a processor, memory system, and the datapath.
  - **significant performance bottlenecks**
- Microprocessor clock speeds gain and High levels of device integration
- How best to utilize these resources?
  - Multiple functional units and execute multiple instructions in the same cycle.

**Implicitly parallel** if its compiler or interpreter can recognize opportunities for parallelization and implement them without being told to do so.

All microprocessors currently available rely on parallelism to varying degrees. These are generally hidden from the programmer

## Implicitly parallelism

- Pipelining
- VLIW Processor and etc

<https://bheemhh.github.io/#/HPC26>

# Current Trends in Architecture

29

- Cannot continue to leverage Instruction-Level parallelism (ILP)
  - Single processor performance improvement ended in 2003
- New models for performance:
  - Data-level parallelism (DLP)
  - Thread-level parallelism (TLP)
  - Request-level parallelism (RLP)
- These require explicit restructuring of the application

# Limitations of Memory System Performance

30

- Limitations of Memory System Performance
  - Bandwidth and Latency

- **Improving Effective Memory Latency Using Caches**
  - **memory acts as a low-latency high-bandwidth storage**
  - **piece of data is repeatedly used, the effective latency of this memory system can be reduced by the cache.**
  - **Cache hit ratio achieved by a code on a memory system often determines its performance.**
  - **Exploiting spatial and temporal locality** in applications is critical for amortizing memory latency and increasing effective memory bandwidth.

Typical computations only run at 10-50% of peak CPU utilization because of memory bottlenecks. The key question here is how to connect a 50 ns latency memory to a processor that runs a 0.5 ns clock

Consider a processor operating at 1 GHz (1 ns clock) connected to a DRAM with a latency of 100 ns (no caches). Assume that the processor has two multiply-add units and is capable of executing four instructions in each cycle of 1 ns. The following observations follow: